



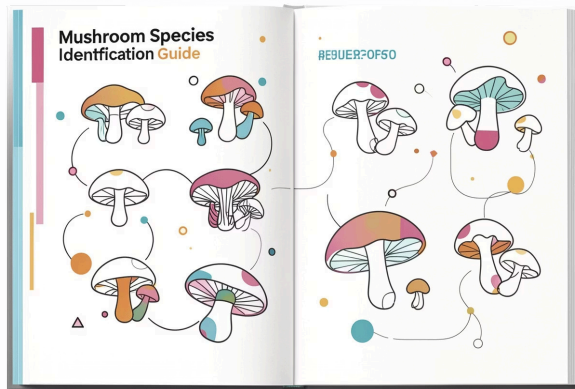
Mushrooms: Edible vs Poisonous

A complete Machine Learning project — from UCI dataset to a containerised FastAPI prediction service. Three models, one pipeline, zero guesswork.

ML · FASTAPI · DOCKER

UCI DATASET #73

The Dataset



UCI Machine Learning Repository — Dataset #73

A classic **binary classification** task: given a mushroom's categorical attributes, predict whether it is **edible (e)** or **poisonous (p)**.

Features

Categorical attributes encoded as single-letter codes — cap-shape, odor, habitat, gill-size, and more

Missing Values

The stalk-root field may contain ? for unknown values, handled during preprocessing

Target

p → 1 (poisonous), e → 0 (edible)

Preprocessing Pipeline

A custom `DataProcessorTransformer` wraps all cleaning logic into a scikit-learn compatible transformer — fully pipeline-safe and serialisable.



Impute Missing

Encode Features

Remove Zero-Variance

Map Target

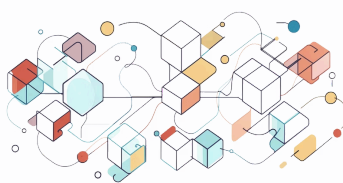
Using `handle_unknown="ignore"` ensures unseen categories at inference time become all-zero columns — no crashes, no surprises in production.

Three Models, One Goal



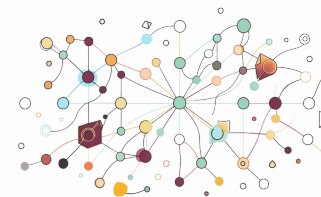
Logistic Regression

A transparent, interpretable baseline. Fast to train, easy to debug — ideal for establishing a performance floor and inspecting feature coefficients.



XGBoost

Gradient-boosted decision trees via `XGBClassifier`. Typically the strongest performer on tabular categorical data with minimal tuning required.



Neural Network

A small MLP with dense layers and dropout, trained via `SciKeras` `KerasClassifier`. Early stopping prevents overfitting; also evaluated on ROC-AUC.

Training & Evaluation

Each pipeline — preprocessing plus model — is trained, evaluated, and serialised independently with `joblib` for zero-friction deployment.

Accuracy

Primary metric across all three models for direct comparison

ROC-AUC

Additional evaluation for the Neural Network, capturing discrimination quality

Classification Report

Precision, recall, and F1 per class — especially useful for the NN

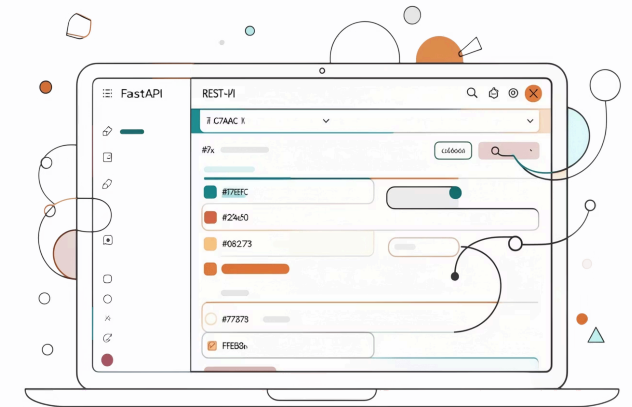
joblib Export

Full pipelines saved as `.joblib` files — preprocessing and model bundled together

FastAPI Service

Three Prediction Endpoints

The API loads each serialised pipeline at startup and exposes a dedicated endpoint per model. Requests are validated via a **Pydantic model** with field aliases (e.g. `cap-shape`), then converted to a single-row DataFrame for prediction.



GET /

Health check

POST /predict/lr

Logistic Regression

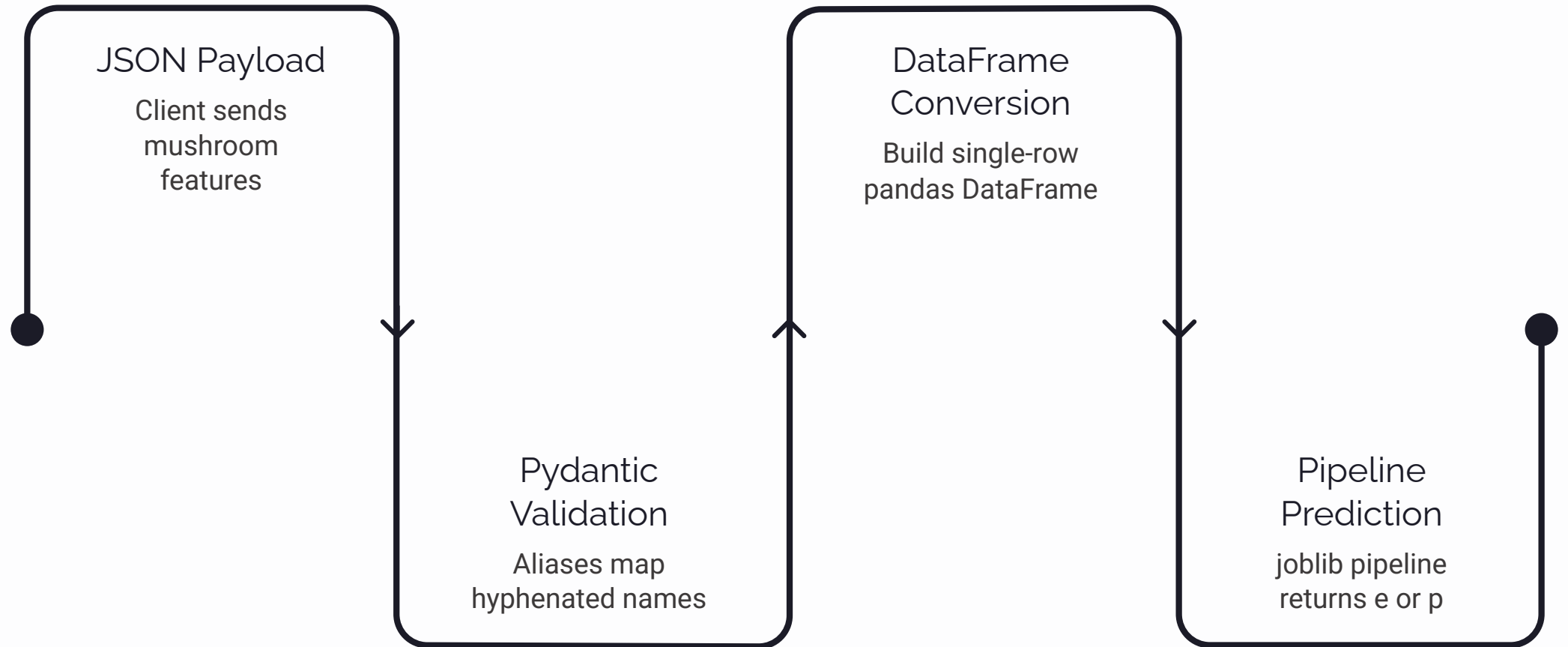
POST /predict/xgb

XGBoost

POST /predict/nn

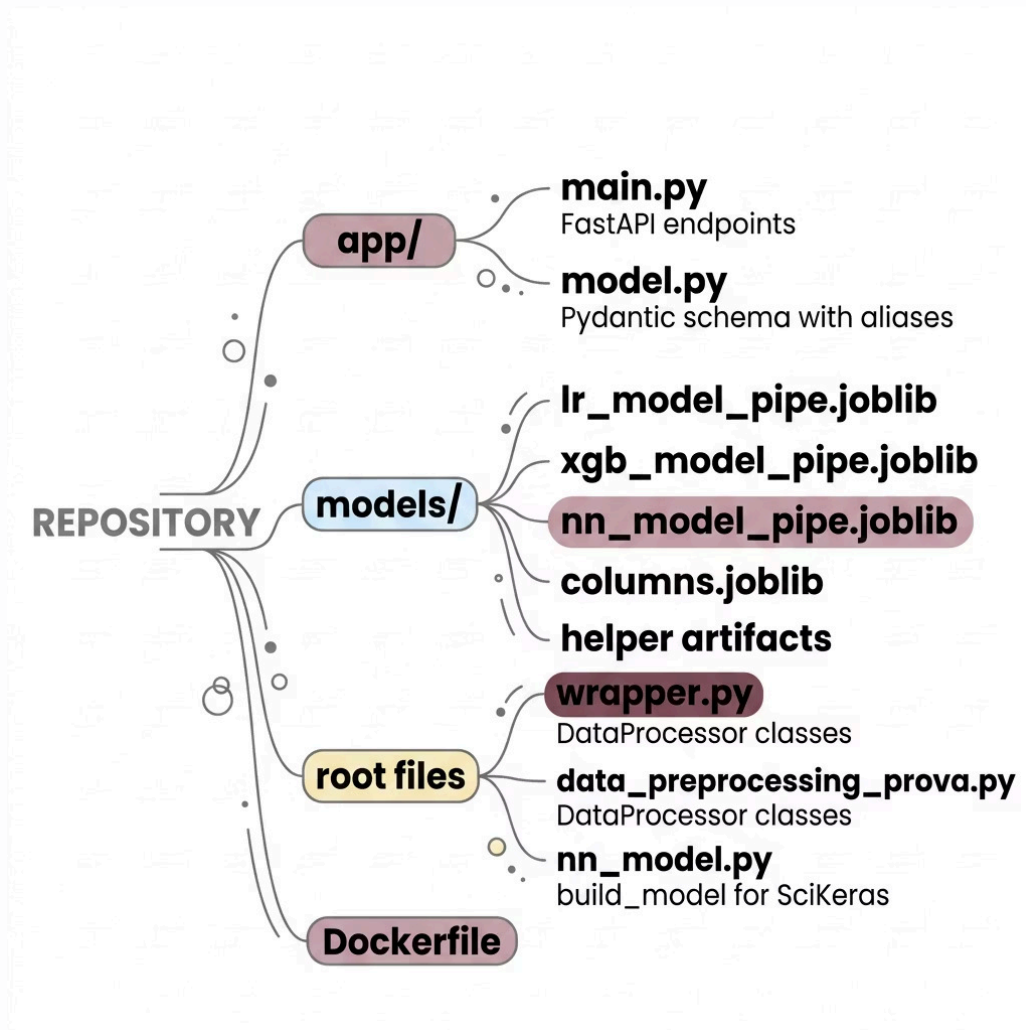
Neural Network

Request → Prediction Flow



Every step is deterministic and reproducible. The same preprocessing applied at training time is replayed at inference — no manual feature engineering required from the client.

Repository Structure



Key Files at a Glance

The project is organised for clarity — separation of concerns between API logic, model artifacts, and preprocessing utilities.

- `app/main.py` — FastAPI router with all `/predict/*` endpoints
- `app/model.py` — Pydantic schema with hyphenated-field aliases
- `nn_model.py` — `build_model()` required for SciKeras deserialisation
- `wrapper.py` — `DataProcessor` and `DataProcessorTransformer`

Key Takeaways

1 End-to-end ML pipeline

From raw UCI data to a live, containerised prediction API – fully reproducible and production-ready.

2 Robust preprocessing by design

Custom transformer, `handle_unknown="ignore"`, and `VarianceThreshold` ensure the pipeline handles real-world inference gracefully.

3 Model diversity for comparison

Logistic Regression, XGBoost, and a Keras Neural Network – three approaches, one consistent evaluation framework.

4 Docker for zero-friction deployment

One image, one command. The service runs identically on any machine with Docker installed.

